



中山大學  
SUN YAT-SEN UNIVERSITY

## 信息安全技术课程实验报告

基于论文《A Watermark for Large Language Models》的复现与改进研究

姓名 \_\_\_\_\_

学号 \_\_\_\_\_

学院 \_\_\_\_\_

专业 \_\_\_\_\_

2026 年 6 月

## 摘 要

本实验围绕大语言模型文本水印技术展开，选择 ICML 2023 论文《A Watermark for Large Language Models》作为复现对象，并在官方固定 **delta** 水印方案的基础上，设计了一种基于熵的自适应 **delta** 文本水印改进方法。实验首先对官方仓库进行完整复现，验证其在本地环境下能够稳定完成无水印文本生成、带水印文本生成与统计检测；随后在保留原始 **greenlist** 划分机制和检测框架不变的前提下，将固定偏置替换为随当前 token 概率分布熵动态变化的偏置，从而实现水印强度的自适应调节。正式实验统一使用 **opt-125m** 模型、12 条英文 prompts，并从 **z-score**、**green\_fraction**、检测成功率以及文本多样性等角度，对无水印方案、固定 **delta** 方案和自适应 **delta** 方案进行了批量对比。结果表明，固定大偏置方法虽然在检测强度上最强，但会明显降低文本质量；而自适应 **delta** 方法能够在保持较高水印检测能力的同时，缓解强固定偏置带来的重复增加和多样性下降问题，更适合作为一种“质量—检测均衡型”的课程改进方案。

**关键词：**大语言模型；文本水印；论文复现；自适应 **delta**；信息安全

# 目录

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>实验背景和动机</b>                        | <b>1</b>  |
| <b>2</b> | <b>实验目的</b>                           | <b>1</b>  |
| <b>3</b> | <b>原文复现</b>                           | <b>2</b>  |
| 3.1      | 论文方案概述 . . . . .                      | 2         |
| 3.2      | 复现环境与项目组织 . . . . .                   | 2         |
| 3.3      | 原始复现步骤 . . . . .                      | 3         |
| 3.4      | 复现结果与验证 . . . . .                     | 5         |
| <b>4</b> | <b>实验改进</b>                           | <b>5</b>  |
| 4.1      | 改进问题分析 . . . . .                      | 5         |
| 4.2      | 改进方案设计 . . . . .                      | 6         |
| 4.3      | 代码实现与工程改进 . . . . .                   | 6         |
| 4.4      | 改进方案原理分析 . . . . .                    | 7         |
| <b>5</b> | <b>实验验证和结果分析</b>                      | <b>8</b>  |
| 5.1      | 实验设置 . . . . .                        | 8         |
| 5.2      | 对比实验结果 . . . . .                      | 10        |
| 5.3      | 结果分析 . . . . .                        | 10        |
| 5.4      | 综合实验结果 . . . . .                      | 12        |
| <b>6</b> | <b>遇到的问题 and 解决</b>                   | <b>12</b> |
| 6.1      | 问题一：官方前端与新版 Gradio 接口不兼容 . . . . .    | 12        |
| 6.2      | 问题二：模型下载与本地运行环境配置不稳定 . . . . .        | 12        |
| 6.3      | 问题三：原始仓库只提供固定 delta 的交互演示，不适合做系统性课程比较 | 13        |
| <b>7</b> | <b>实验总结和心得</b>                        | <b>13</b> |
| <b>A</b> | <b>改进方案源代码</b>                        | <b>13</b> |

# 1 实验背景和动机

大语言模型在内容生成、问答辅助、教育支持、编程协作等场景中已经表现出很强的实用价值，但与此同时，模型输出内容的可追踪性与可验证性问题也逐渐成为信息安全领域的重要研究方向。一方面，大模型生成文本在形式上越来越接近人工撰写内容，如果缺少额外标记，平台、教师、企业和监管者很难快速判断某段文本是否由模型生成；另一方面，生成式模型被广泛接入搜索、写作和社交平台之后，伪造信息、自动批量灌水、学术不端和内容归属不清等问题也更加突出。在这样的背景下，文本水印技术成为连接生成模型与内容治理的重要安全机制。与传统密码学中的加密或签名不同，文本水印不要求直接暴露密钥，也不要求改变文本的外部显示格式，而是通过对生成过程施加概率偏置，使得文本在统计意义上带有可检测的“生成痕迹”。因此，围绕文本水印开展复现和改进，不仅契合《信息安全技术》课程中关于数字水印的主题， also 具有很强的现实应用价值。

本实验选择复现并改进 ICML 2023 论文《A Watermark for Large Language Models》，主要动机有三点。第一，该论文是大语言模型文本水印领域最具代表性的早期工作之一，方法清晰、公开代码完整、检测指标明确，非常适合作为课程复现对象。第二，原论文采用固定水印强度参数 `delta` 的方式，在不同生成情形下可能存在“过弱导致检测不稳”或“过强导致文本质量下降”的矛盾，这为改进提供了明确切入点。第三，课程考查要求不仅要完成方案复现，还要体现创新改进和实验验证，因此在成功跑通官方代码的基础上，进一步提出一种更适合课程实验规模的“基于熵的自适应 `delta` 文本水印方法”，并通过批量实验、可视化前端和结果分析脚本来验证改进效果，具有较强的完整性与可展示性。

## 2 实验目的

本实验的目的如下：

1. 选择“数字水印”主题相关的论文《A Watermark for Large Language Models》，完成论文复现与改进型课程项目。
2. 对论文公开代码仓库进行完整复现，验证官方固定 `delta` 文本水印方法在本地环境下可以稳定运行，并正确完成文本生成与水印检测。
3. 在保留原始复现仓库的基础上，设计并实现一种基于熵的自适应 `delta` 改进方案，使其能够根据生成时的分布不确定性动态调整水印强度。
4. 在统一实验设置下，从检测能力与文本质量两个方面，对“无水印”“固定 `delta` 水印”“自适应 `delta` 水印”进行标准评价指标下的对比实验。

5. 形成完整的课程报告、改进代码、批量实验脚本、结果分析脚本、README 文档以及可演示前端。

## 3 原文复现

### 3.1 论文方案概述

论文《A Watermark for Large Language Models》的核心思想是：在语言模型生成每一个 token 之前，根据已有前缀通过伪随机方式把词表划分成 greenlist 和 redlist 两部分，并对 greenlist 中的 token logits 统一增加一个正偏置  $\delta$ 。这样一来，模型在采样时更倾向于从 greenlist 中选择 token，于是最终生成文本中会包含更多符合该随机划分规则的 token。检测阶段使用同样的 seeding 规则和词表划分方式，统计文本中 green token 的比例，并进一步计算 z-score 和 p-value，判断该文本是否显著偏离自然未加水印文本的分布，从而完成“是否带有水印”的统计检验。

从实现上看，官方代码主要由两部分构成。第一部分是 WatermarkLogitsProcessor，它继承自 Hugging Face 的 LogitsProcessor，负责在生成阶段动态计算当前前缀对应的 greenlist，并对 greenlist token 加上固定偏置  $\delta$ 。第二部分是 WatermarkDetector，负责在检测阶段按照相同规则重建 greenlist，统计 num\_green\_tokens、green\_fraction、z\_score、p\_value 等指标，并根据阈值输出 prediction。

### 3.2 复现环境与项目组织

本实验使用官方仓库 jwkirchenbauer/lm-watermarking 作为复现基础。为了避免改进代码污染原始复现结果，项目被组织为两个独立目录：lm-watermarking 用于保留官方仓库及最小改动后的原始复现版本，lm-watermarking-adaptive 用于在保留官方基础代码的前提下，实现课程要求的改进方案、批量实验脚本、分析脚本和可视化前端。

正式实验中使用的主要环境和资源如下：

1. 操作系统环境：本地 Windows + PowerShell。
2. 模型运行框架：PyTorch 与 Transformers。
3. 演示前端：Gradio。
4. 本地模型目录：models/。
5. 实验提示词文件：course\_project/data/prompts.txt。
6. 实验结果输出目录：course\_project/outputs/。

7. 模型选择策略：为了先验证流程正确性，先使用体积较小的 `tiny-gpt2` 进行 smoke test；在批量实验阶段，为了获得更稳定的文本统计结果，使用 `opt-125m` 作为正式实验模型。

本实验的项目组织结构示意如下：

```
lm-watermarking/  
|-- README.md  
|-- requirements.txt  
|-- demo_watermark.py  
|-- app.py  
|-- watermark_processor.py  
|-- extended_watermark_processor.py  
|-- normalizers.py  
|-- alternative_prf_schemes.py  
|-- homoglyphs.py  
|-- homoglyph_data/  
|-- hf_hub_space_demo/  
|-- experiments/  
|-- watermark_reliability_release/  
`-- models/  
    |-- tiny-gpt2/
```

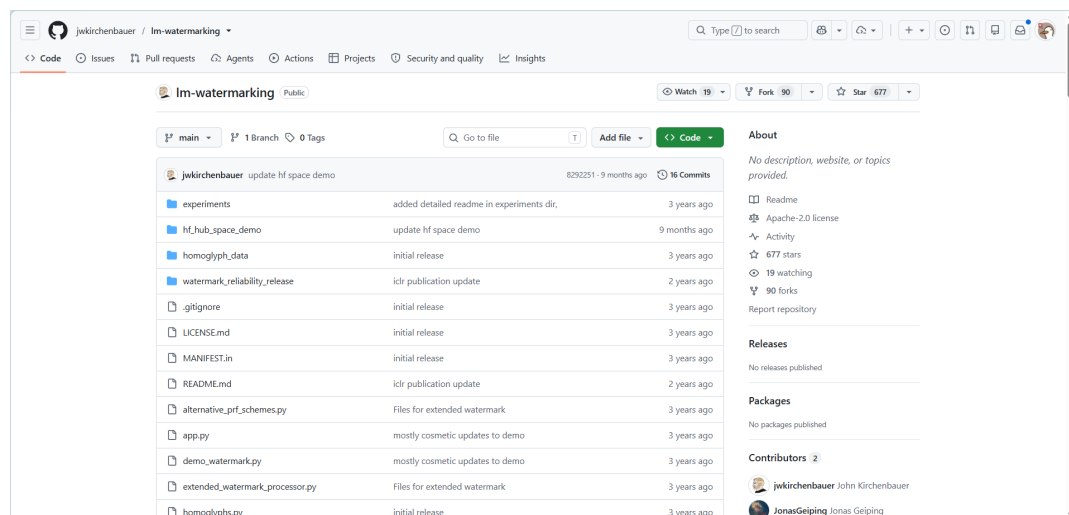


图 1: 官方仓库下载与环境配置截图

### 3.3 原始复现步骤

原始复现按照“环境准备—依赖安装—模型下载—官方 Demo 运行—结果验证”的顺序完成，具体步骤如下。

1. 从 GitHub 克隆官方仓库，作为后续复现和改进的基础：

```
git clone https://github.com/jwkirchenbauer/lm-watermarking.git
```

2. 创建实验环境并安装依赖：

```
conda create -n inform python=3.10 -y
conda activate inform
pip install -r requirements.txt
```

3. 准备模型与下载环境。由于官方仓库依赖 Hugging Face 模型下载，实验过程中对代理和本地缓存目录进行了配置，以确保模型可以成功下载到 models/ 下。为了先验证功能链路是否正常，实验首先准备了较小规模的 tiny-gpt2 模型用于测试。

4. 运行官方 Gradio Demo：

```
python demo_watermark.py \
  --model_name_or_path ./models/tiny-gpt2 \
  --use_gpu True \
  --run_gradio True \
  --max_new_tokens 100
```

5. 处理兼容性问题并完成验证。在复现过程中发现，官方代码中的 `demo.queue(concurrency_count=3)` 与较新版本的 Gradio 不兼容，因此将其改为 `demo.queue(default_concurrency_limit=3)`，从而恢复前端页面的正常启动。

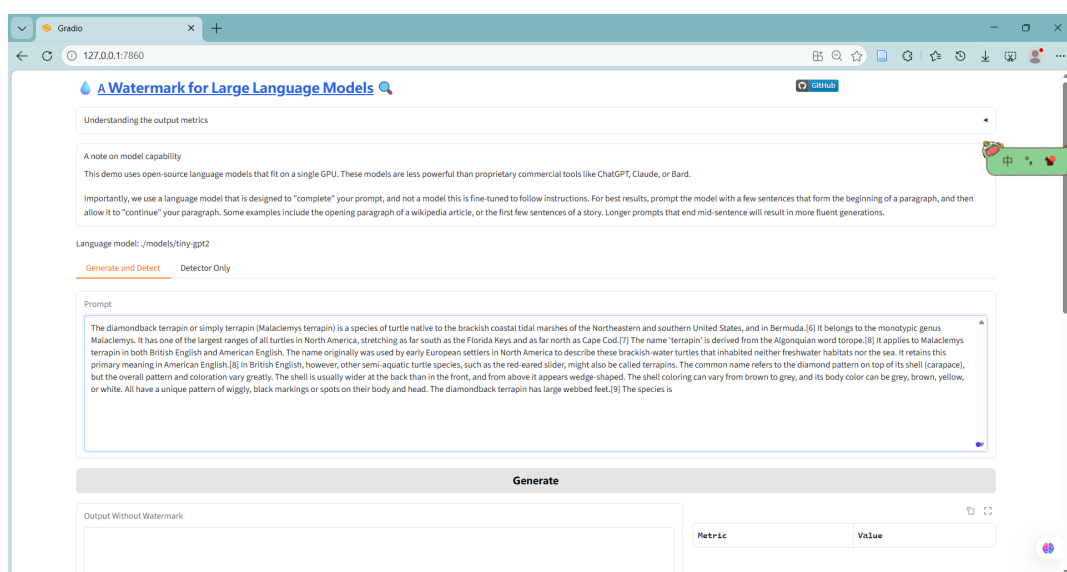


图 2: 官方复现运行成功截图

### 3.4 复现结果与验证

在官方 Demo 中，同一输入提示词可以同时生成无水印文本和有水印文本，并在右侧直接显示检测结果。一次典型实验中，无水印文本的 `green_fraction` 约为 22.2%，`z-score`=-0.638，检测结果为 `Human/Unwatermarked`；而有水印文本的 `green_fraction` 约为 70.4%，`z-score`=10.4，检测结果为 `Watermarked`，且置信度接近 100%。这一现象与论文结论一致，即带水印文本在 `green token` 占比和统计显著性上明显高于普通生成文本。

复现结果说明：官方固定 `delta` 的方法在本地环境中是可运行、可检测和可解释的；检测器使用的统计量能够有效区分“未加水印文本”和“带水印文本”；原论文提出的 `greenlist` 偏置思路在不修改模型参数的情况下即可完成嵌入与检测。这为后续改进实验提供了可靠基线。

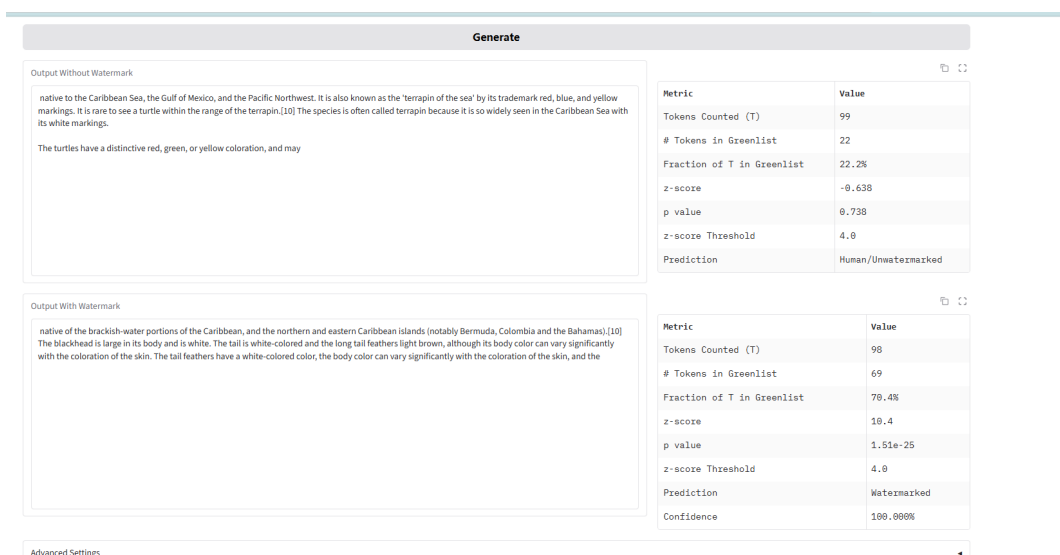


图 3: 官方 Gradio Demo 运行界面与检测结果截图

## 4 实验改进

### 4.1 改进问题分析

在阅读论文与运行官方 Demo 的过程中，可以发现原方法的一个关键控制参数是 `delta`，即对 `greenlist token` 施加的统一 logits 偏置。固定 `delta` 的优点是实现简单、可解释性强，但缺点也很直接：如果 `delta` 设置过小，模型虽然被轻微引导到 `greenlist` 上，但统计痕迹不够明显，检测 `z-score` 可能不足；如果 `delta` 设置过大，虽然可以显著提高 `green token` 比例并提升检测成功率，但生成过程会过度偏向特定 `token` 子集，进而可能带来表达单一、重复增加、自然度下降等副作用。因此，固定 `delta` 本质上反映的是“检测能力”和“文本质量”之间的静态折中，而不是一种上下文自适应的控制策略。



本实验的改进思路是：生成过程本身并不是均匀不变的，不同位置的 next-token 分布不确定性不同。当模型已经非常确定下一个 token 应该是什么时，继续施加强水印可能会不必要地改变自然生成路径；当模型本身存在多种合理候选、分布熵较大时，适度增强水印偏置则更容易嵌入统计痕迹，同时对语义质量的破坏可能更小。因此，水印强度不应固定不变，而应当根据当前分布的不确定性动态调节。

## 4.2 改进方案设计

基于上述观察，本文设计了“基于熵的自适应 delta 文本水印方法”。其核心思想是在每一步生成时，不再直接使用固定偏置 delta，而是先对当前 logits 做 softmax 得到 token 概率分布，再计算该分布的信息熵，并将熵归一化到  $[0, 1]$  区间，最后得到当前步的自适应偏置：

$$\delta_t = \delta_{\min} + (\delta_{\max} - \delta_{\min}) \cdot H_{\text{norm}}$$

其中， $H_{\text{norm}}$  表示当前 next-token 分布的归一化熵，delta\_min 和 delta\_max 分别表示允许的最小水印强度和最大水印强度。当模型越不确定时，说明候选 token 更分散，此时  $\delta_t$  越接近 delta\_max；当模型越确定时， $\delta_t$  越接近 delta\_min。

需要强调的是，本实验并没有改变官方方法中 greenlist 的划分机制，也没有修改检测器的统计原理，改进方案只替换了“对 greenlist token 施加固定偏置”这一局部操作，而把官方方法中最核心的 seeded partition 机制全部保留。

## 4.3 代码实现与工程改进

改进方案对应的目录结构示意图如下：

```
lm-watermarking-adaptive/  
|-- demo_watermark.py  
|-- watermark_processor.py  
|-- requirements.txt  
|-- models/  
|   |-- tiny-gpt2/  
|   `-- opt-125m/  
`-- course_project/  
    |-- data/  
    |   `-- prompts.txt  
    |-- docs/  
    |   `-- README_SUBMISSION.md  
    |-- outputs/  
    |   |-- results.csv  
    |   |-- summary.md
```

```
| |-- zscore_comparison.png
| |-- green_fraction_comparison.png
|-- processors/
| |-- adaptive_watermark_processor.py
|-- scripts/
| |-- run_experiments.py
| |-- analyze_results.py
|-- demo_adaptive.py
```

1. `course_project/processors/adaptive_watermark_processor.py`: 实现 `AdaptiveDeltaWatermark`
2. `course_project/scripts/run_experiments.py`: 实现批量实验脚本。
3. `course_project/scripts/analyze_results.py`: 实现结果分析脚本。
4. `course_project/scripts/demo_adaptive.py`: 实现课程演示前端。

原始官方仓库负责“保持复现真实性”，`course_project` 负责“承载课程改进与实验展示”。

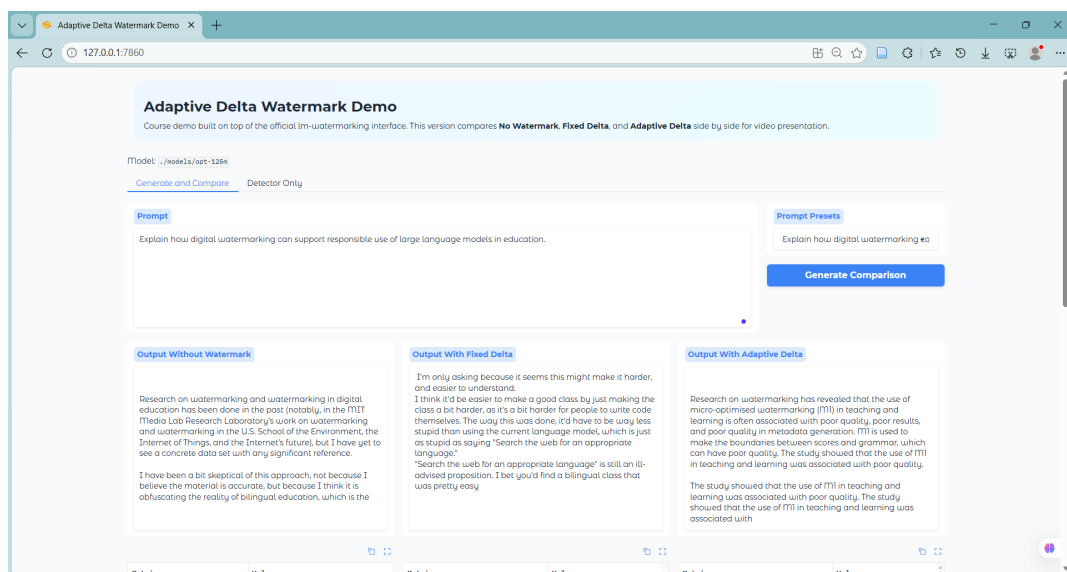


图 4: 改进版三方案对比前端界面截图

## 4.4 改进方案原理分析

从原理上看，自适应 `delta` 方案试图解决的是“水印强度静态配置”与“语言模型生成动态变化”之间的不匹配。固定 `delta` 相当于假设每一步生成的可干预强度都应该一样，但实际上，语言模型在不同上下文位置的分布形态差异很大。有些位置本来就是高置信度 `token`，如果继续施加强偏置，很可能把本来就集中分布的结果进一步推向局

部模式；另一些位置本身候选分布宽、熵大，说明可选择空间更多，此时适当增加水印强度，往往可以在不显著破坏语义的前提下增加可检测性。

因此，自适应 `delta` 可以理解为一种“基于分布状态的软控制”机制：它不重新训练模型，不引入外部监督，而是直接利用模型当下给出的分布信息来自我调节水印强度。

表 1: 原方案与改进方案共同点对比

| 比较维度 | 共同说明                                                    |
|------|---------------------------------------------------------|
| 共同目标 | 通过提高 greenlist token 的采样概率，在生成文本中嵌入可检测的统计水印             |
| 共同基础 | 使用相同的 seeding 机制、greenlist 划分方式和 WatermarkDetector 检测框架 |

表 2: 原方案与改进方案差异点对比

| 比较维度        | 官方固定 <code>delta</code> 水印方案                               | 基于熵的自适应 <code>delta</code> 水印方案                                                      |
|-------------|------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 水印强度控制方式    | 每一步都使用固定偏置 <code>delta</code>                              | 每一步根据当前分布熵动态计算 <code>delta_t</code>                                                  |
| 参数形式        | 单一固定参数，例如 <code>delta=2.0</code>                           | 区间参数 <code>delta_min</code> 、 <code>delta_max</code> ，并结合 <code>H_norm</code> 计算实时偏置 |
| 对上下文变化的适应能力 | 较弱，不区分当前生成位置的分布不确定性                                        | 较强，能够根据不同位置的分布状态调节偏置强弱                                                               |
| 检测能力特点      | <code>delta</code> 越大通常 <code>z-score</code> 越高，但可能带来更强副作用 | 检测能力略低于最强固定偏置方案，但总体仍能保持较高检出率                                                         |
| 对文本质量的影响    | 强偏置下更容易出现重复增加、多样性下降                                        | 通过在高确定位置减小偏置，力图缓解文本质量下降问题                                                            |
| 可解释性        | 方法简单直接，参数含义清晰                                              | 既保留原始可解释性，又能记录实际使用过的 <code>delta</code> 范围                                           |

## 5 实验验证和结果分析

### 5.1 实验设置

为了对原方案和改进方案进行公平比较，本实验对模型、提示词、生成方式和检测参数进行了统一设置，具体如表3所示。

表 3: 实验设置

| 项目                    | 具体内容                                                                                        |
|-----------------------|---------------------------------------------------------------------------------------------|
| 正式实验模型                | 本地 opt-125m                                                                                 |
| 提示词来源                 | course_project/data/prompts.txt                                                             |
| 提示词数量                 | 12 条                                                                                        |
| 提示词主题                 | 数字水印、信息安全、网络安全、教育、人工智能治理等                                                                   |
| 对比方法                  | No Watermark、Fixed Delta 0.5、Fixed Delta 1.0、Fixed Delta 2.0、Fixed Delta 3.0、Adaptive Delta |
| 随机控制                  | 相同随机种子下生成各方案结果                                                                              |
| gamma                 | 0.25                                                                                        |
| max_new_tokens        | 100                                                                                         |
| detection_z_threshold | 4.0                                                                                         |
| 自适应方案参数               | delta_min=0.5, delta_max=3.0                                                                |

为了保证对比结果既包含检测性能，也反映文本质量，本实验选择如下指标作为标准评价指标。

表 4: 评价指标说明

| 指标                     | 含义                                  |
|------------------------|-------------------------------------|
| green_fraction         | 生成文本中 green token 所占比例，越高通常说明水印痕迹越强 |
| z-score                | 核心检测统计量，越高表示越显著偏离无水印分布              |
| p-value                | 在原假设下观测到该统计量的概率，越小越支持“带水印”          |
| prediction             | 在阈值 $z=4.0$ 下的最终分类结果                |
| Detection Success Rate | 在一组 prompt 上被判定为 Watermarked 的比例    |
| Distinct-1             | 不同 unigram 占总词数比例，用于衡量词汇多样性         |
| Distinct-2             | 不同 bigram 占总 bigram 数比例，用于衡量短语多样性   |
| Repetition Rate        | 重复率，数值越低通常表示重复更少                    |

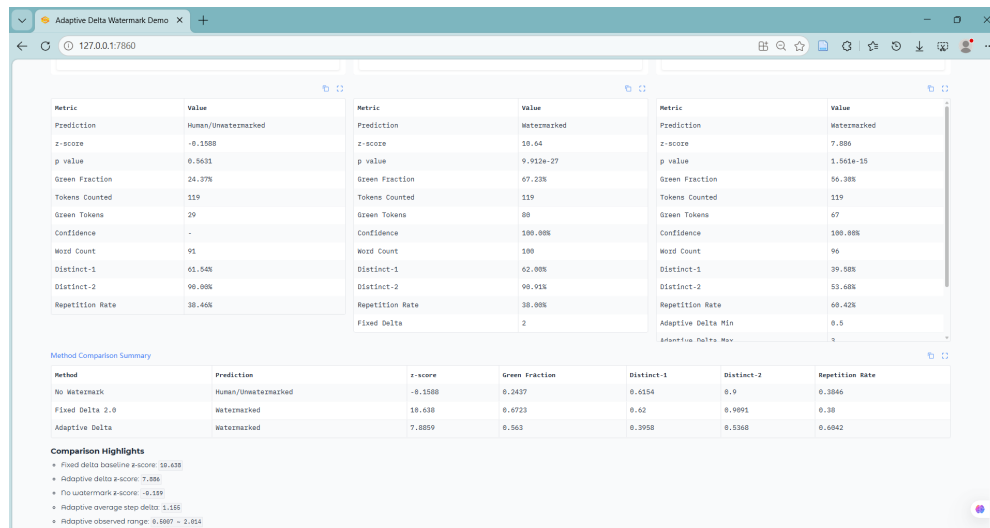


图 5: 改进方案检测指标展示截图

## 5.2 对比实验结果

批量实验得到的平均结果如表5所示。

表 5: 各方案平均结果对比

| 方法              | 样本数 | 平均<br>z-score | 平均<br>green<br>frac-<br>tion | 检测成<br>功率 | 平均<br>Distinct-<br>1 | 平均<br>Distinct-<br>2 | 平均<br>Repeti-<br>tion<br>Rate |
|-----------------|-----|---------------|------------------------------|-----------|----------------------|----------------------|-------------------------------|
| No Watermark    | 12  | 0.5292        | 0.2729                       | 0.00%     | 0.6143               | 0.8491               | 0.3857                        |
| Fixed Delta 0.5 | 12  | 1.6291        | 0.3264                       | 0.00%     | 0.6383               | 0.8590               | 0.3617                        |
| Fixed Delta 1.0 | 12  | 4.4561        | 0.4450                       | 66.67%    | 0.6135               | 0.8461               | 0.3865                        |
| Fixed Delta 2.0 | 12  | 8.6330        | 0.6313                       | 100.00%   | 0.6155               | 0.8185               | 0.3845                        |
| Fixed Delta 3.0 | 12  | 13.2299       | 0.8258                       | 100.00%   | 0.5318               | 0.7171               | 0.4682                        |
| Adaptive Delta  | 12  | 6.1467        | 0.5231                       | 83.33%    | 0.5994               | 0.8238               | 0.4006                        |

此外，对自适应方案单独统计可得，其在全部实验中的平均步长偏置约为 1.207，实际观测到的最小偏置约为 0.5001，最大偏置约为 2.2305。这一结果说明，自适应方法确实没有一直退化为固定大 **delta**，而是根据当前分布状态在较宽范围内动态调整偏置强度。

## 5.3 结果分析

根据表5的实验结果，可以得到如下几点结论：

1. No Watermark 的平均 z-score 仅为 0.5292，检测成功率为 0%，说明检测器不会轻易把普通输出误判为带水印文本。
2. Fixed Delta 0.5 的平均 z-score 只有 1.6291，仍明显低于检测阈值，说明偏置过小时难以形成稳定且可检测的统计水印。
3. 随着固定 delta 的增大，检测能力明显增强。Fixed Delta 2.0 和 Fixed Delta 3.0 的检测成功率都达到 100%。
4. Fixed Delta 3.0 虽然在检测能力上最强，但同时也表现出最明显的文本质量退化，其 Distinct-1 和 Distinct-2 降低、重复率上升。
5. Fixed Delta 2.0 在保证 100% 检测成功率的同时，文本质量指标明显优于 Fixed Delta 3.0，因此可以视为本实验中较强且较稳定的官方 baseline。
6. 改进方案 Adaptive Delta 的平均 z-score 为 6.1467，检测成功率达到 83.33%，说明其在大多数样本上都能够形成足够强的统计水印痕迹。
7. 自适应方案的平均步长偏置约为 1.207，实际观测范围在 0.5001 到 2.2305 之间，说明该方法确实根据当前分布状态动态调整了水印强度。
8. 如果同时考虑“检测成功率”和“文本质量保持”两个目标，那么自适应方案相较于强固定偏置方法表现出更合理的权衡关系，因此更适合作为一种“质量—检测均衡型”的课程改进方案。

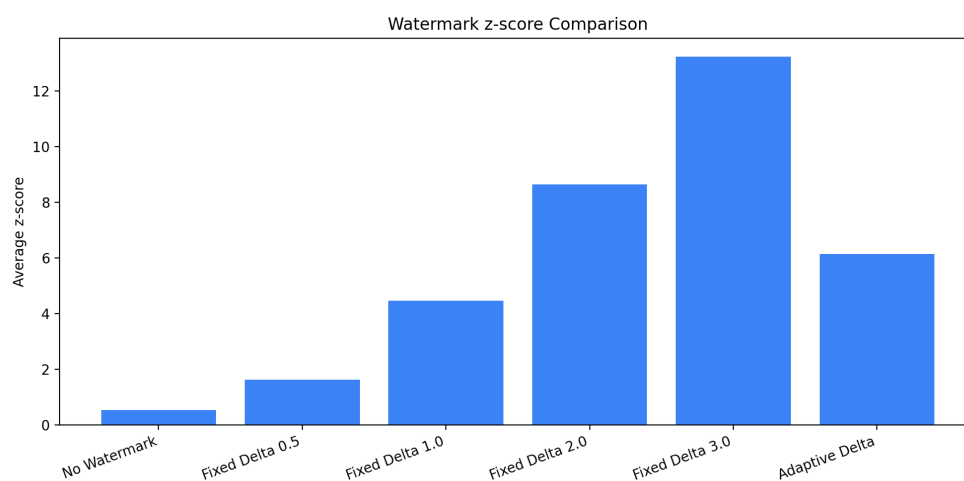


图 6: 不同方案平均 z-score 对比图

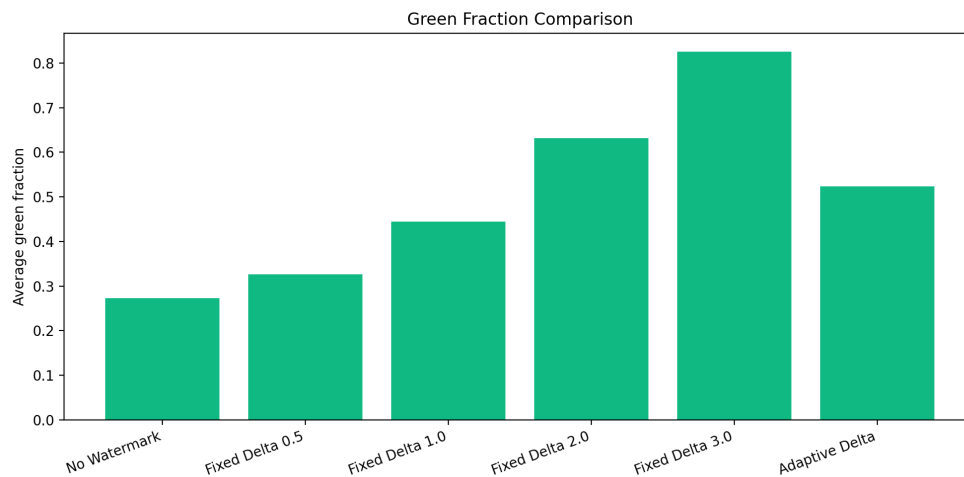


图 7: 不同方案平均 green fraction 对比图

## 5.4 综合实验结果

综合复现、改进与对比实验，本次实验得到如下结果：

1. 官方论文《A Watermark for Large Language Models》的固定 `delta` 文本水印方法已经在本地环境中成功复现，官方 Demo、检测器与文本生成链路均可正常运行。
2. 本文提出的基于熵的自适应 `delta` 改进方案已经完成实现，并且被整合进独立的课程项目结构中，包含批量实验脚本、分析脚本与视频演示前端。
3. 在 12 组 prompts 的正式实验中，自适应方法取得了平均 `z-score`=6.1467、平均 `green_fraction`=0.5231、检测成功率 83.33% 的结果。
4. 对比结果表明：Fixed Delta 3.0 检测能力最强，但文本质量退化更明显；Fixed Delta 2.0 是强有力 baseline；Adaptive Delta 则提供了相对更均衡的方案。

# 6 遇到的问题和解决

## 6.1 问题一：官方前端与新版 Gradio 接口不兼容

在初次运行官方 Demo 时，页面没有正常启动，错误信息显示 `Blocks.queue()` 不再接受 `concurrency_count` 参数。为解决该问题，实验中将 `demo.queue(concurrency_count=3)` 修改为 `demo.queue(default_concurrency_limit=3)`。

## 6.2 问题二：模型下载与本地运行环境配置不稳定

由于实验依赖 Hugging Face 模型下载，早期在下载和加载模型时受到网络与代理配置影响，导致模型拉取失败或速度过慢。为解决这一问题，实验先使用 `tiny-gpt2` 跑

通完整链路，再切换到 opt-125m 完成正式实验。

### 6.3 问题三：原始仓库只提供固定 delta 的交互演示，不适合做系统性课程比较

官方仓库主要面向论文方法演示，只提供固定 delta 的网页交互界面，不方便一次性对多组 prompts、多组 delta 参数和改进方法进行批量实验与结果统计。为解决这一问题，实验在保留原仓库的基础上单独创建了 course\_project 目录，新增批量实验脚本、分析脚本和改进前端。

## 7 实验总结和心得

本次实验从论文阅读、代码复现、工程调试、方法改进到实验分析，较完整地经历了一次“从学术方案到课程项目”的落地过程。通过复现《A Watermark for Large Language Models》，可以直观理解文本水印并不是简单地在输出后附加标记，而是深度嵌入到生成概率分布中的一种统计可检测机制。通过进一步实现基于熵的自适应 delta 方法，又能够体会到“改进创新”并不一定意味着完全推翻原方法，而更常见的是在准确把握原理之后，对其中最关键的局部环节提出合理、可验证、可比较的小步改进。

从个人学习心得来看，这次课程项目最大的收获在于把“读论文”“复现代码”“做实验”和“写报告”真正串联起来。过去阅读论文时，往往更关注结论本身，而这次实验让我认识到，很多论文中的关键贡献只有在亲自复现之后，才能理解其工程边界和实际效果。总体而言，本次实验不仅加深了我对数字水印和大语言模型安全问题的理解，也提升了我将研究型代码整理成可复现实验项目和可展示成果的能力。

## 参考文献

1. Kirchenbauer J, Geiping J, Wen Y, et al. A Watermark for Large Language Models[C]//International Conference on Machine Learning. PMLR, 2023: 17061–17084.
2. 官方代码仓库: <https://github.com/jwkirchenbauer/lm-watermarking>

## A 改进方案源代码

下面给出本实验改进方案 AdaptiveDeltaWatermarkLogitsProcessor 的核心源代码。该代码保留了官方方法的 greenlist 构造逻辑，仅将固定 delta 替换为基于当前 logits 分布熵动态调整的 delta\_t。

```
# coding=utf-8
from __future__ import annotations
```



```
from statistics import mean
from pathlib import Path
import sys

import torch

REPO_ROOT = Path(__file__).resolve().parents[2]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from watermark_processor import WatermarkLogitsProcessor

class AdaptiveDeltaWatermarkLogitsProcessor(WatermarkLogitsProcessor):
    """Apply a per-step watermark bias based on the entropy of the next-token
    distribution."""

    def __init__(self, *args, delta_min: float = 0.5, delta_max: float = 3.0,
    **kwargs):
        super().__init__(*args, **kwargs)
        if delta_min > delta_max:
            raise ValueError("delta_min must be less than or equal to delta_max
            .")

        self.delta_min = delta_min
        self.delta_max = delta_max
        self.delta_history: list[list[float]] = []

    def _compute_step_biases(self, scores: torch.FloatTensor) -> torch.
    FloatTensor:
        log_probs = torch.log_softmax(scores, dim=-1)
        probs = log_probs.exp()
        entropies = -(probs * log_probs).sum(dim=-1)
        max_entropy = scores.new_tensor(float(max(self.vocab_size, 2))).log()
        normalized_entropies = (entropies / max_entropy).clamp(0.0, 1.0)
        return self.delta_min + (self.delta_max - self.delta_min) *
        normalized_entropies

    def get_delta_summary(self) -> dict[str, float | None]:
        flattened = [delta for step in self.delta_history for delta in step]
```

```
    if not flattened:
        return {
            "avg_step_delta": None,
            "observed_delta_min": None,
            "observed_delta_max": None,
        }
    return {
        "avg_step_delta": mean(flattened),
        "observed_delta_min": min(flattened),
        "observed_delta_max": max(flattened),
    }

def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor)
-> torch.FloatTensor:
    if self.rng is None:
        self.rng = torch.Generator(device=input_ids.device)

    batched_greenlist_ids = [None for _ in range(input_ids.shape[0])]
    for batch_index in range(input_ids.shape[0]):
        batched_greenlist_ids[batch_index] = self._get_greenlist_ids(
input_ids[batch_index])

    green_tokens_mask = self._calc_greenlist_mask(scores=scores,
greenlist_token_ids=batched_greenlist_ids)
    step_biases = self._compute_step_biases(scores).to(scores.dtype)
    self.delta_history.append(step_biases.detach().cpu().tolist())

    bias_matrix = green_tokens_mask.to(scores.dtype) * step_biases.
unsqueeze(-1)
    return scores + bias_matrix
```